

CourseHub — Modelo Relacional

Levantado via `information_schema` do banco `coursehub_escola` em 2026-08-02, após a migration `20260802_001_drop_activity_submissions_and_staff_role` . 28 tabelas confirmadas. Nenhuma coluna, relação ou tabela abaixo foi inferida — tudo foi lido diretamente do schema live.

1. Como ler este documento

Cada tabela lista: colunas relevantes (tipo, nulidade, default quando não-óbvio), chaves únicas (incluindo compostas), e chaves estrangeiras com sua regra `ON DELETE` . Comentários de "por quê" aparecem quando a regra de negócio não é óbvia só olhando o schema.

2. Identidade e acesso

users

Tabela raiz de autenticação. Todo `students` / `teachers` tem exatamente um `users` correspondente (1:1 via `user_id`).

Coluna	Tipo	Notas
<code>id</code>	<code>int PK</code>	
<code>name</code> , <code>email</code>	<code>varchar(150)</code>	<code>email</code> UNIQUE
<code>password_hash</code>	<code>varchar(255)</code>	<code>bcrypt</code>
<code>gender</code>	<code>varchar(30)</code>	nullable, texto livre
<code>avatar_key</code>	<code>varchar(50)</code>	nullable
<code>role</code>	<code>enum('admin','teacher','student')</code>	Reduzido de 5 para 3 valores em 2026-08-02 — <code>manager</code> e <code>staff</code> removidos por não terem nenhuma implementação de aplicação (ver ADR-001)
<code>status</code>	<code>enum('active','inactive','blocked')</code>	

refresh_tokens

Coluna	Tipo	Notas
id	bigint unsigned PK	
user_id	int, FK → users.id ON DELETE CASCADE	
token_hash	char(64) UNIQUE	SHA-256 do token opaco — o valor bruto nunca é persistido
expires_at , revoked_at	datetime	revoked_at nullable
replaced_by_token_hash	char(64)	nullable — rastreia rotação (defesa contra replay de token roubado)

password_reset_tokens

Mesmo padrão de hash de refresh_tokens . FK → users.id ON DELETE CASCADE. token_hash UNIQUE. Janela de validade de 15 minutos (definida em código, não no schema).

3. Perfis (1:1 com users)

students

FK user_id → users.id UNIQUE, ON DELETE CASCADE. Duplica name / email / gender de users (padrão do projeto: perfil "denormalizado" para leitura direta sem join). Campos próprios: registration_number UNIQUE, birth_date , cpf UNIQUE, phone , address , status enum('active','inactive','graduated','cancelled').

teachers

Mesmo padrão de students . Campos próprios: registration_number UNIQUE, cpf UNIQUE, phone , specialty , status enum('active','inactive').

A tabela staff (mesmo padrão, para o papel staff) foi **removida** em 2026-08-02 — zero linhas, zero código a referenciava.

4. Catálogo acadêmico

`courses`

Coluna	Tipo	Notas
<code>id</code>	<code>int PK</code>	
<code>teacher_id</code>	<code>int, FK → teachers.id ON DELETE SET NULL</code>	Nullable de propósito — um curso pode existir sem professor responsável (comportamento esperado, não bug; ver <code>business-rules.md</code>)
<code>name</code> , <code>description</code> , <code>expanded_description</code> , <code>syllabus</code>	<code>texto</code>	
<code>workload_hours</code>	<code>int</code>	
<code>price</code>	<code>decimal(10,2)</code>	
<code>status</code>	<code>enum('active','inactive','draft','archived')</code>	
<code>image_url</code> , <code>nivel</code> , <code>category</code>	<code>varchar</code>	
<code>planned_session_count</code>	<code>int</code>	<code>nullable</code> — quantidade de encontros planejados para turmas do curso

`course_pricing_plans`

FK `course_id` → `courses.id` (sem regra `ON DELETE` explícita no schema atual). Define condições de cobrança-base do curso: `billing_type` `enum('one_time','monthly_plan')`, `total_amount` , `monthly_payment_count` , `monthly_payment_amount` , `max_card_installments` , `accepts_pix` / `accepts_boleto` / `accepts_credit_card` (`bool`), `status` .

classes (turmas)

FK course_id → courses.id ON DELETE CASCADE; FK teacher_id → teachers.id ON DELETE RESTRICT (não é possível apagar um professor com turmas ativas). shift
enum('morning','afternoon','night','online'), start_date / end_date , status
enum('active','inactive','finished').

class_sessions (encontros de uma turma)

FK class_id → classes.id ON DELETE CASCADE. session_number + class_id UNIQUE
composta (unique_class_session_number). session_type
enum('class','review','exam','presentation','workshop','lab','recovery','other'). status
enum('scheduled','completed','cancelled','archived') — **nota**: cancelled (cancelamento pelo professor) e archived (remoção lógica) são estados distintos, ambos soft; nenhum DELETE físico ocorre nesta tabela pelo código de aplicação atual.

course_contents

FK course_id → courses.id ON DELETE CASCADE; FK class_id → classes.id ON DELETE RESTRICT, **nullable**. class_id IS NULL = conteúdo geral do curso (visível a todas as turmas);
class_id = X = exclusivo daquela turma. type
enum('video','pdf','text','live_class','activity','assessment'). due_date opcional (usado pelo agregador de calendário quando presente).

5. Matrícula e progresso

enrollments

FK student_id → students.id ON DELETE CASCADE; course_id → courses.id ON DELETE CASCADE; class_id → classes.id ON DELETE SET NULL, nullable. UNIQUE composta (student_id, course_id) — **um aluno não pode ter duas matrículas ativas no mesmo curso**.
status enum('active','inactive','completed','cancelled','locked') — locked é usado pela régua de cobrança (lock_reason enum('financial_overdue','administrative','academic','other')), com
locked_at / locked_by_user_id / lock_note e o par simétrico reactivated_at / reactivated_by_user_id .

student_content_progress

FK student_id → students.id , course_id → courses.id , content_id → course_contents.id ,
todas ON DELETE CASCADE. UNIQUE composta (student_id, content_id) . status

enum('not_started','in_progress','completed'), progress_percentage , last_position_seconds (retomar vídeo).

attendance

FK class_session_id → class_sessions.id ON DELETE CASCADE; student_id → students.id ON DELETE RESTRICT. UNIQUE composta (class_session_id, student_id) . status enum('present','absent','late','excused').

6. Atividades e avaliações

activities

FK course_id → courses.id ON DELETE CASCADE; class_id → classes.id ON DELETE RESTRICT, nullable — **mesmo padrão de escopo geral/turma de course_contents** . activity_kind enum('activity','exam') — diferencia "atividade" de "avaliação" na interface, mesma tabela para ambas. type enum('mixed','quiz','text','upload'). max_score decimal(5,2) default 10.00. status enum('active','inactive','draft','archived').

activity_questions / activity_options

activity_questions : FK activity_id → activities.id ON DELETE CASCADE. question_type enum('multiple_choice','text','upload'). points decimal(5,2).
activity_options : FK question_id → activity_questions.id ON DELETE CASCADE. is_correct boolean.

submissions / submission_answers / grades

submissions : FK activity_id → activities.id ON DELETE CASCADE; student_id → students.id ON DELETE CASCADE; graded_by_teacher_id → teachers.id ON DELETE SET NULL. UNIQUE composta (student_id, activity_id) — **um envio por aluno por atividade** (correções substituem o envio existente, não criam um novo). status enum('draft','submitted','pending_review','graded','returned').

submission_answers : FK submission_id → submissions.id ON DELETE CASCADE; question_id → activity_questions.id ON DELETE CASCADE; option_id → activity_options.id ON DELETE SET NULL (se uma alternativa for removida, a resposta histórica não desaparece, só perde a referência). UNIQUE composta (submission_id, question_id) .

`grades` : FK `submission_id` → `submissions.id` ON DELETE CASCADE, UNIQUE (`uk_grade_submission`); `student_id` / `course_id` / `activity_id` também referenciados (CASCADE), `teacher_id` → `teachers.id` ON DELETE SET NULL. UNIQUE composta adicional (`student_id`, `activity_id`) (`uk_grade_student_activity`) — reforça a mesma regra de "uma nota por aluno por atividade" no nível da tabela de notas oficiais.

`activity_submissions` — tabela paralela a `submissions` + `grades`, com propósito sobreposto (mesmo conceito: envio de aluno com nota/feedback/status embutidos). Confirmada sem nenhuma linha e sem nenhuma referência em código antes de ser removida em 2026-08-02 — o mecanismo real e único de submissão/correção é `submissions` + `submission_answers` + `grades`.

7. Financeiro

`financial_contracts`

FK `enrollment_id` → `enrollments.id` UNIQUE (`uq_financial_contract_enrollment`) — **1:1 com a matrícula**; `pricing_plan_id` → `course_pricing_plans.id`. Espelha os campos de cobrança de `course_pricing_plans` no momento da contratação (congelamento de condições). `status` enum('pending','active','overdue','completed','cancelled') — recalculado a partir do estado das faturas por `services/financial/contractFinancialService.js`, nunca setado diretamente pelo cliente.

`invoices`

FK `financial_contract_id` → `financial_contracts.id`. UNIQUE composta (`financial_contract_id`, `installment_number`) (`uq_contract_installment`). `invoice_type` enum('full_payment','monthly_payment'). `status` enum('pending','processing','paid','overdue','cancelled','refunded'). Suporta desconto (`discount_amount`, `discount_reason`, auditoria de quem aplicou).

`payments`

FK `invoice_id` → `invoices.id`. UNIQUE composta (`gateway`, `gateway_payment_id`). `source` enum('gateway','admin_manual','system') — pagamentos manuais registrados por admin usam `gateway`: "simulated" fixo (ver known issue sobre `simulatedGateway.js` em `system-overview.md`). `status` enum('created','pending','approved','rejected','cancelled','refunded','chargeback'). Campos condicionais por método: `card_installments` / `card_brand` / `card_last_four` (cartão), `pix_copy_paste` / `pix_qr_code` / `pix_expires_at` (Pix), `boleto_barcode` / `boleto_url` / `boleto_due_date` (boleto).

payment_events / financial_events

Duas trilhas de auditoria distintas: `payment_events` (FK `payment_id`, UNIQUE `gateway_event_id`) registra mudanças de status de um pagamento específico; `financial_events` (FKs opcionais para `financial_contract_id`, `invoice_id`, `payment_id`, `enrollment_id`) é a trilha genérica de qualquer alteração financeira administrativa (`event_type` livre, `previous_value` / `new_value` em JSON, `source` enum('system','admin','gateway','student'), `actor_user_id`).

invoice_collection_actions

FK `invoice_id` → `invoices.id`. UNIQUE composta (`invoice_id`, `action_type`). `action_type` enum com 6 estágios: `reminder_3_days_before`, `due_date_notice`, `marked_overdue`, `overdue_charge_10_days`, `lock_warning_15_days`, `enrollment_locked_30_days`. `status` enum('pending','processed','failed','skipped').

Nenhum código de aplicação cria ou processa linhas nesta tabela. O schema modela uma régua de cobrança automatizada completa, mas não há job, rota ou service que a execute hoje. Tratar como **funcionalidade planejada, não implementada** — não inventar um mecanismo de execução que não existe.

8. Calendário

academic_calendar_events

Única tabela própria do módulo de calendário — os outros tipos de evento (atividades com `due_date`, conteúdos com `due_date`, sessões de turma) são agregados por leitura a partir de suas tabelas de origem, nunca duplicados aqui (ver `docs/diagrams/calendar-flow.md`). FK `class_id` → `classes.id` ON DELETE RESTRICT, `course_id` → `courses.id` ON DELETE RESTRICT, `created_by_user_id` → `users.id` ON DELETE RESTRICT, todos nullable/condicionais ao `scope_type`. `event_type` enum com 10 valores institucionais (feriado, recesso, semana de provas, matrícula, etc.). `scope_type` enum('institutional','all_students','all_teachers','course','class') controla quem vê o evento. `status` enum('active','cancelled') — soft delete via `cancelled_at`.

9. Planejadas, não implementadas

certificates

Schema completo (`certificate_code` UNIQUE, `workload_hours` , `final_score` , `status` enum('issued','cancelled')), FKs para `students` / `courses` ON DELETE CASCADE. **Zero código de aplicação a referencia.** A tela `EmissaoAdmin.jsx` (emissão de certificados) existe no frontend mas usa dados mockados localmente, sem chamar nenhuma API — não há endpoint de certificados no backend.

declarations

Schema completo (`declaration_type` enum('enrollment','attendance','completion','custom'), `title` , `body` , `status` enum('draft','issued','cancelled')). **Zero código de aplicação a referencia.**

10. Convenções observadas em todo o schema

- Toda tabela de negócio tem `created_at` / `updated_at` `datetime` com `DEFAULT_GENERATED` (e `ON UPDATE CURRENT_TIMESTAMP` em `updated_at`).
- **Nenhuma exclusão física é praticada pelo código de aplicação** em nenhuma das entidades centrais (`students` , `teachers` , `courses` , `classes` , `class_sessions` , `activities` , `enrollments` , `invoices` , `payments`) — todas usam soft delete via coluna `status` . As únicas exclusões físicas confirmadas no código são de registros efêmeros: tokens revogados/expirados não são limpos automaticamente (ficam retidos), e não há rotina de purga observada.
- MySQL `DATE` / `DATETIME` são lidos pelo driver `mysql2` no fuso horário **local do processo Node**, não UTC — o backend nunca usa `toISOString()` para formatar essas colunas (usa getters locais); ver `utils/appConfig.js` e o padrão replicado no frontend (`utils/dateUtils.js`).

11. Histórico de migrations

Não existe histórico de migrations anterior a 2026-08-02 neste repositório — nenhum arquivo `.sql` de versionamento de schema foi encontrado em nenhuma branch antes dessa data; o schema evoluiu diretamente no banco. A partir de

`database/migrations/20260802_001_drop_activity_submissions_and_staff_role.sql` (com rollback correspondente em `database/rollback/`), toda alteração de schema passa a ser versionada. Este documento reflete o estado confirmado via `information_schema` nessa mesma data — os

`CREATE_TIME` de cada tabela (não reproduzidos aqui) sugerem que a maior parte do schema foi criada em 2026-07-24, com extensões pontuais em 2026-07-25, 07-29, 07-31 e 08-01 (calendário).